Fully-Distributed Dynamic Packet Routing for LEO Satellite Networks: A GNN-Enhanced Multi-Agent Reinforcement Learning Approach

Yongyi Ran, *Member, IEEE*, Yajie Ding, Shuangwu Chen, *Member, IEEE*, Jizhao Lei, and Jiangtao Luo, *Senior Member, IEEE*

***Abstract*—An efficient routing strategy in Low Earth Orbit (LEO) satellite networks is critical for air-space-ground integrated communication towards 6G. However, the existing terrestrial routing algorithms cannot well handle the high-speed movement issue of satellites, while the existing satellite routing algorithms usually suffer from high communication overhead or high computational complexity. To address the above issues, we propose a fully distributed dynamic packet routing algorithm based on Graph Neural Network (GNN)-enhanced Multi-Agent Deep Reinforcement Learning (MADRL), named GraphPR. In GraphPR, the satellite routing problem is modeled as a Partially Observable Markov Decision Process (POMDP), where each satellite only needs to share the information with one-hop neighbors. Then, Graph Attention Network (GAT) is leveraged to encode the perceived one-hop information and derive a hidden representation implicitly consisting of multi-hop satellite information. Subsequently, MADRL is employed to build a fully distributed optimization framework and make routing decisions. In addition, a special mechanism called Residual Shortest Path Hops (RSPH) is designed to guide the routing selection and avoid routing loops. Finally, the experimental results illustrate that GraphPR has better performance in terms of packet loss rate, average delivery time, throughput, and average queuing length than the baseline algorithms.**

***Index Terms*—LEO satellite networks, fully distributed routing, graph attention network, deep reinforcement learning.**

I. Introduction

Satellite networks, especially with the advancements in Low Earth Orbit (LEO) satellites, are increasingly regarded as a vital complement to terrestrial networks in 6G communication systems. However, the LEO satellites are constantly in high-speed motion, which introduces dynamic changes in the network topology, precarious Inter-Satellite Links (ISLs), frequent handovers from one coverage area to another, etc. These dynamic factors prevent the traditional routing strategies designed for terrestrial networks from being directly applied in LEO satellite networks. To gain more benefits from LEO satellite communication, designing a dynamic and efficient routing strategy tailored to the specific characteristics of satellite-based communication systems is of great significance.

However, designing a dynamic and efficient routing algorithm for LEO satellite networks faces a set of challenges. First, a large number of satellites and ISLs, coupled with dynamic characteristics such as satellite position, ISL transmission rate, and network load, would result in much higher routing computational complexity, and even a "curse of dimensionality". Second, constructing a global perspective by collecting the states of all satellites would lead to high communication overhead, while using the states of partial satellites to calculate routing strategies would result in local optimization. Third, due to the high-speed movement of satellites, the topology of the LEO network changes frequently, which requires the routing algorithms to have a strong generalization ability, otherwise, the routing performance would be degraded.

The existing routing approaches of LEO satellite networks can be roughly divided into centralized routing and distributed routing according to where the routing decisions are made. **For the centralized routing approaches**, DT-TTAR [1] divides the time-varying network topology into a set of snapshots and collects global link information for each snapshot to calculate the optimal path. EEDRL-Dijkstra [2] utilizes an energy-efficient event-driven Deep Reinforcement Learning (DRL) enhanced Dijkstra's algorithm to optimize the energy consumption while satisfying the end-to-end delay constraint. [3], [4] leverage software-defined network architecture to dynamically collect all ISL states to a central controller and then make routing decisions centrally. **For the distributed routing approaches**, OPSPF [5] utilizes the regularity of constellations to generate periodic routing tables and deals with irregular link changes via flooding the changes across the network. ASER [6] groups satellites into areas to maintain a relatively static inter-area routing and reduce routing overhead. Both FDR-MARL [7] and DQN-IR [8] use a satellite's local network features to make routing decisions based on Multi-Agent Deep Reinforcement Learning (MADRL). **In summary**, the above centralized approaches can make optimal routing decisions according to the global view, but suffer from excessive communication overhead and computational complexity especially when the LEO satellite constellation is very large. On the contrary, the distributed approaches can well reduce the computing and communication overhead, but they are prone to being trapped in local optimization due to local and partial observations. To address these issues, we will investigate a distributed algorithm that can make better routing decisions just by sharing one-hop information.

In this paper, we propose a fully-distributed dynamic packet routing algorithm for LEO satellite networks based on Graph Attention Network (GAT)-enhanced MADRL, named GraphPR. The main contributions can be summarized as follows: 1) In order to alleviate the communication overhead, each satellite only perceives information from its one-hop adjacent satellites, and then we model the optimization problem as a Partially Observable Markov Decision Process (POMDP). 2) To overcome the issue of local optimization and improve the generalization ability of the proposed GraphPR, we leverage GAT [9] to encode and share the perceived one-hop information, and finally derive a hidden representation implicitly consisting of multi-hop satellite information. 3) To reduce the routing computational complexity and better capture the dynamic characteristics, we employ MADRL to make routing decisions in a fully distributed paradigm. 4) We design a special mechanism called Residual Shortest Path Hops (RSPH) to avoid routing loops which could be easily caused by the natural mesh structure of LEO

Received 21 January 2024; revised 21 July 2024 and 24 September 2024; accepted 10 November 2024. Date of publication 18 November 2024; date of current version 5 March 2025. This work was supported in part by the National Natural Science Foundation of China under Grant U23A20275, Grant 62101525, and Grant 62171072, in part by the Natural Science Foundation of Chongqing under Grant cstc2021jcyj-msxmX0586, and in part by the Science and Technology Program Project of Yubei District in Chongqing (2021-18-nongshe). The review of this article was coordinated by Dr. Lin X. Cai. *(Corresponding authors: Shuangwu Chen; Jiangtao Luo.)*

Yongyi Ran, Yajie Ding, and Jiangtao Luo are with the School of Communication and Information Engineering, Chongqing University of Posts and Telecommunications, Chongqing 400065, China (e-mail: ranyy@cqupt.edu.cn; 1347393696@qq.com; luojt@cqupt.edu.cn).

Shuangwu Chen is with the School of Information Science and Technology, University of Science and Technology of China, Hefei 230027, China (e-mail: chensw@ustc.edu.cn).

Jizhao Lei is with the China Satellite Network Group Company, Ltd., Chongqing 401147, China (e-mail: xdljz2000@163.com).

Digital Object Identifier 10.1109/TVT.2024.3499933

0018-9545 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Yonsei Univ. Downloaded on April 10,2026 at 02:15:54 UTC from IEEE Xplore. Restrictions apply.

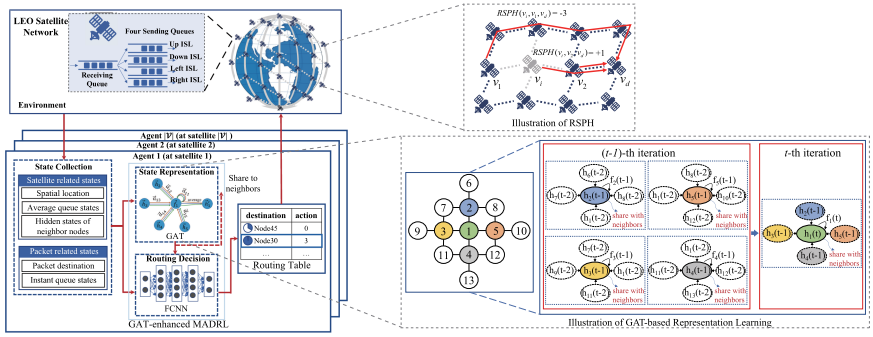

Fig. 1. The system architecture of our proposed GraphPR.

satellite networks. 5) Finally, extensive experiments are carried out and the results illustrate that the proposed GraphPR has better performance in terms of packet loss rate, average delivery time, throughput, and average queuing length than the baseline algorithms.

II. Problem Formulation

A. System Architecture

As illustrated in Fig. 1, the system architecture includes two parts: LEO satellite network environment and GraphPR agent.

LEO Satellite Network Environment: This part consists of a LEO satellite constellation, which has N orbits (orbital altitude is H and inclination is θ) and M satellites per orbit. The LEO satellite network can be modeled as a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ is the set of satellites in the constellation, the total number of satellites can be represented as $|\mathcal{V}| = N \times M$, $\mathcal{E}$ is the set of ISLs, and $e_{i,j} \in \mathcal{E}$ denotes the ISL between satellite v_i and v_j. Each satellite can establish four ISLs, except being located in polar regions or cross seam, including two intra-plane ISLs and two inter-plane ISLs. In addition, each satellite maintains one receiving queue and four sending queues corresponding to four ISLs.

GraphPR Agent: Each satellite in the network would be treated as a GraphPR Agent, which includes the following modules. The **State Collection Module** collects the satellite related states and packet related states. The satellite related states include satellite spatial location, average queue states, and hidden states of neighbor satellites, while the packet related states contain packet destination and instant queue states. The **State Representation Module** feeds the satellite related states into a GAT and outputs a hidden representation (i.e. hidden state). The **Routing Decision Module** generates routing entries by utilizing a Fully-connected Neural Network (FCNN). The routing entries are stored in the routing table and will be deleted when the survival time becomes zero.

B. Communication Model

Ignoring the effects of gravitation, solar activity, etc., the free space propagation loss of ISL between satellite v_i and v_j at time slot t can be approximated by:

$$L_{i,j,t} = \left(\frac{4\pi||v_{i,t}v_{j,t}||f}{c}\right)^2 \tag{1}$$

where f is the carrier frequency, c denotes the speed of light, $||v_{i,t}v_{j,t}|| = \sqrt{(x_{i,t}-x_{j,t})^2+(y_{i,t}-y_{j,t})^2+(z_{i,t}-z_{j,t})^2}$ denotes the Euclidean distance between $v_{i,t}$ and $v_{j,t}$, $(x_{i,t}, y_{i,t}, z_{i,t})$ and $(x_{j,t}, y_{j,t}, z_{j,t})$ is the coordinate of $v_{i,t}$ and $v_{j,t}$ respectively. Assume that the bit error rate $P_b = 10^{-7}$ and the modulation mode is 8-PSK, then the bit energy-to-noise density ratio $\mathbb{E}_b/\mathbb{N}_0$ can be obtained by $P_b = \frac{1}{3}erfc(\sqrt{\frac{3\mathbb{E}_b}{\mathbb{N}_0}}\sin\frac{\pi}{8})$, where $\mathbb{E}_b$ denotes energy per bit, $\mathbb{N}_0$ denotes the noise power spectral density and $erfc$ is complementary error function. Then the transmission rate from v_i to v_j at time slot t is [10]:

$$R_{i,j,t} = \frac{\mathbb{C}/\mathbb{N}}{\mathbb{E}_b/\mathbb{N}_0}\cdot B = \frac{P_t G_t G_r}{L_{i,j,t}k_B T}\cdot\frac{1}{\mathbb{E}_b/\mathbb{N}_0} \tag{2}$$

where $\mathbb{C}$ is the carrier power, $\mathbb{N}$ is the noise power, P_t is the transmit power, G_t and G_r are transmit and receive antenna gains, k_B denotes the Boltzmann constant, B denotes the channel bandwidth and T denotes the thermal noise.

C. Queue Model

As shown in Fig. 1, the newly arrived packet at a satellite will be firstly cached in the receiving queue and then forwarded to the corresponding sending queue based on the routing strategy. If the corresponding sending queue is full or the packet's Time To Live (TTL) is zero, the packet will be discarded. All queues follow the principle of first-in-first-out.

For each sending queue φ on satellite v_i, the queue length at the beginning of time slot t is evolved as:

$$q_{i,\varphi,t} = \min\{q_{i,\varphi,t-1} + z_{i,\varphi,t-1} - o_{i,\varphi,t-1}, q_{\max}\} \tag{3}$$

where $z_{i,\varphi,t-1}$ and $o_{i,\varphi,t-1}$ are the number of packets received and sent by the sending queue φ during the time slot $t-1$, respectively, and $q_{\max}$ is the maximum queue length.

The agent collects the length values of all sending queues at each time slot t and calculates the average values for all sending queues per Δ time slots. Then, the average queue length of sending queue φ on satellite v_i ranging from time slot $t-\Delta+1$ to time slot t can be expressed as:

$$q^{\text{avg}}_{i,\varphi,t} = \left(\sum_{\iota=t-\Delta+1}^{t} q_{i,\varphi,\iota}\right)/\Delta \tag{4}$$

D. Delay Model

The propagation delay of a packet transmitted between v_i and v_j can be expressed as:

$$\tau^{\eta}_{i,j,t} = ||v_{i,t}v_{j,t}||/c \tag{5}$$

Assume that the process of packet arrival and processing in the sending queues follows the $M/M/1/q_{\max}$ queuing model with arrival rate λ and

Authorized licensed use limited to: Yonsei Univ. Downloaded on April 10,2026 at 02:15:54 UTC from IEEE Xplore. Restrictions apply.

service rate μ. If the packet k is transferred into the sending queue φ of v_i at time slot t, and will be forwarded to satellite v_j, then the queuing delay of packet k can be calculated as:

$$\tau^q_{i,k,t} = \frac{\lambda}{\mu(\mu-\lambda)} = W_s\rho \approx q^{\text{avg}}_{i,\varphi,t} \cdot \frac{\vartheta}{R^{\text{avg}}_{i,j,t}} \tag{6}$$

where $W_s = 1/(\mu-\lambda)$ represents the mean waiting time and $\rho = \lambda/\mu$, ϑ is the packet size, $R^{\text{avg}}_{i,j,t}$ denotes the average transmission rate from v_i to v_j during the packet k being blocked in the sending queue.

E. Problem Formulation

In this paper, we aim to find the optimal path for each packet to minimize average end-to-end delivery time while keeping the packet loss rate low. Suppose that the forwarding path of packet k between the source satellite and the target satellite can be expressed as $\mathcal{D}_k = (\mathcal{V}'_k, \mathcal{E}'_k)$, where $\mathcal{V}'_k$ and $\mathcal{E}'_k$ denotes the set of satellites and links through which the packet k passed, and $q^w_{i,k}$ is the queue length where packet k waits at satellite v_i. Then, the objective function is expressed as:

$$\begin{aligned}
&\min_{\mathcal{D}} \frac{1}{|\mathcal{K}|}\sum_{k\in\mathcal{K}}\left(\lambda_1\sum_{v_i\in\mathcal{V}'_k}\tau^q_{i,k,t} + \lambda_2\sum_{e_{i,j}\in\mathcal{E}'_k}\tau^\eta_{i,j,t}\right)\\
&\text{s.t.}\quad \text{C1}: \mathcal{V}'_k \subseteq \mathcal{V}, \mathcal{E}'_k \subseteq \mathcal{E}, |\mathcal{V}'_k| < H_{\max}\\
&\qquad \text{C2}: ||v_{i,t}v_{j,t}|| \le I_{i,j,t}, \forall e_{i,j} \in \mathcal{E}\\
&\qquad \text{C3}: q^w_{i,k} < q_{\max}, \forall v_i \in \mathcal{V}'_k\\
&\qquad \text{C4}: \lambda_1 + \lambda_2 = 1
\end{aligned} \tag{7}$$

where $\mathcal{K}$ is the set of packets, $H_{\max}$ represents the maximum hops, $I_{i,j,t}$ indicates the Line of Sight (LoS) distance between v_i and v_j, λ_1 and λ_2 are the weights of queuing delay and propagation delay, respectively. The constraint $C1$ implies that the number of network satellites through which the packet passes must be less than the maximum hops, $C2$ means that the length of ISL between satellite v_i and v_j should not exceed the LoS distance, as the ISL can only be established when there are no obstructions between the two satellites. $C3$ means that the maximum queue length of each satellite is $q_{\max}$, and $C4$ denotes that the sum of the weight factors of queuing delay and propagation delay is 1.

III. The Proposed GraphPR Algorithm

A. GAT Based Representation Learning

In order to reduce communication overhead, overcome local optimization problems caused by local observation, and improve the generalization ability of GraphPR, we leverage the GNN to aggregate and extract features of the LEO satellite network. GAT is a variant of GNN that implements the attention mechanism to calculate the importance of different neighbor satellites to the current satellite. If the input feature of current satellite v_i to GAT is $f_i \in \mathbb{R}^F$, the input feature of neighbor satellite v_j is $h_j \in \mathbb{R}^F$, the output feature of v_i from GAT is $h'_i \in \mathbb{R}^{F'}$, the attention coefficient of v_j to v_i is:

$$\alpha_{i,j} = \frac{e^{\zeta[\mathbf{a}^T(\mathbf{W}f_i||\mathbf{W}h_j)]}}{\sum_{u\in\mathcal{N}_i} e^{\zeta[\mathbf{a}^T(\mathbf{W}f_i||\mathbf{W}h_u)]} + e^{\zeta[\mathbf{a}^T(\mathbf{W}f_i||\mathbf{W}f_i)]}} \tag{8}$$

where $\mathbf{W} \in \mathbb{R}^{F'\times F}$ is a weight matrix, $\mathbf{a} \in \mathbb{R}^{2F'}$ is a weight vector, ζ denotes LeakyReLU activation function, $\mathcal{N}_i$ is the set of neighbor satellites of v_i, $\cdot^T$ denotes transposition and $||$ is the concatenation operation. Thus the output feature h'_i is:

$$h'_i = \sigma\left(\sum\nolimits_{j\in\mathcal{N}(i)} \alpha_{i,j}\mathbf{W}h_j + \alpha_{i,i}\mathbf{W}f_i\right) \tag{9}$$

where σ denotes an activation function.

The illustration of GAT-based Representation Learning in Fig. 1, taking the satellite v_1 as an example, at the t-th iteration, GAT will aggregate the current satellite states $f_1(t)$ with the hidden states shared by the neighbor satellites located in the up, down, left, right directions, which are denoted as $h_2(t-1)$, $h_4(t-1)$, $h_3(t-1)$ and $h_5(t-1)$. Then, GAT will output the hidden state $h_1(t)$ for the current satellite v_1 and share it with v_1's neighbor satellites. By sharing many times, the hidden states from the neighbors will implicitly contain multi-hop satellite information.

Let $\mathcal{G}^{-i}$ denote the graph $\mathcal{G}$ without v_i, $SH(v_i, v_d; \mathcal{G})$ denote the shortest path hops from v_i to v_d in $\mathcal{G}$ and $SH(v_j, v_d; \mathcal{G}^{-i})$ denote the shortest path hops from v_j to v_d in $\mathcal{G}^{-i}$. Assume that the current forwarding satellite is v_i, v_j is its next-hop satellite and v_d is the destination. The residual shortest path hops $RSPH(v_i, v_j, v_d)$ can be defined as:

$$RSPH(v_i, v_j, v_d) = SH(v_i, v_d; \mathcal{G}) - SH(v_j, v_d; \mathcal{G}^{-i}) \tag{10}$$

The illustration of RSPH is shown in Fig. 1, satellite v_1 and v_2 are the candidate next-hop satellites of v_i, and $SH(v_i, v_d, \mathcal{G}) = 2$, $SH(v_1, v_d, \mathcal{G}^{-i}) = 5$, $SH(v_2, v_d, \mathcal{G}^{-i}) = 1$. If we select v_1 as the next-hop satellite, then $RSPH(v_i, v_1, v_d) = -3$. If we select v_2 as the next-hop satellite, then $RSPH(v_i, v_2, v_d) = +1$. Only comparing the RSPH, v_2 is better than v_1 as the next-hop satellite. RSPH will be utilized to guide the routing exploration when training our proposed GraphPR algorithm.

B. The Proposed GraphPR Based on MADRL

To reduce the communication overhead, only the above mentioned GAT-encoded hidden states will be shared between satellites. Therefore, we model the dynamic packet routing problem as a POMDP and solve it with MADRL in a fully distributed paradigm.

State Space: The state space of agent i can be expressed as $s_i = < s^n_i, s^p_i >$. $s^n_i = \{f_i, h_{up(i)}, h_{down(i)}, h_{left(i)}, h_{right(i)}\}$ represents the satellite related states, which will be periodically updated and fed into GAT to extract the hidden state h'_i. $f_i = \{\varrho_i, \varsigma_i, \boldsymbol{\chi}_i, \boldsymbol{q}^{\text{avg}}_i\}$ denote the local states of the current satellite v_i, where ϱ_i is the orbit index to which satellite v_i belongs, ς_i is the index of satellite v_i on the orbit ϱ_i, $\boldsymbol{\chi}_i = (x_i, y_i, z_i)$ denotes the satellite coordinate, $\boldsymbol{q}^{\text{avg}}_i = < q^{\text{avg}}_{i,1}, q^{\text{avg}}_{i,2}, q^{\text{avg}}_{i,3}, q^{\text{avg}}_{i,4} >$ indicates the average queue length of four sending queues, respectively. $h_{up(i)}, h_{down(i)}, h_{left(i)}$ and $h_{right(i)}$ correspond to the hidden state of the neighbor satellite in the up, down, left, right directions, respectively. $s^p_i = \{\varrho_d, \varsigma_d, \boldsymbol{\chi}_d, \boldsymbol{q}_i\}$ represent the packet related states, which will be collected when the packet reaches the current satellite, used to concatenate with the hidden state h'_i of the current satellite, and then fed into FCNN to output routing decision, where ϱ_d is the orbit index to which the target satellite v_d belongs, ς_d is the index of the target satellite v_d on the orbit ϱ_d, $\boldsymbol{\chi}_d = (x_d, y_d, z_d)$ denotes the coordinate of the target satellite and $\boldsymbol{q}_i = < q_{i,1}, q_{i,2}, q_{i,3}, q_{i,4} >$ indicates the instant queue length of four sending queues of the current satellite v_i.

Action Space: For each satellite, a data packet can be forwarded to the adjacent satellites located in the up, down, left, and right directions, so the action space $A = \{up, down, left, right\}$.

Reward Function: Based on (7), the reward function of agent i is calculated as:

$$r_{i,t} = \begin{cases} \omega_1 \cdot RSHP(v_i, a_{i,t}, v_d) - \omega_2 \cdot \tau^q_{i,k,t} - \omega_3 \cdot \tau^\eta_{i,j,t}, \\ \qquad \text{if } a_{i,t} \neq v_d, H_k > 0, q^w_{i,k,t} < q_{\max} \\ \Psi, \qquad \text{if } a_{i,t} = v_d, H_k > 0, q^w_{i,k,t} < q_{\max} \\ \Phi, \qquad \text{otherwise} \end{cases} \tag{11}$$

Authorized licensed use limited to: Yonsei Univ. Downloaded on April 10,2026 at 02:15:54 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: The Training Procedure of the Proposed GraphPR.

Input: Initialize initial exploration parameter ϵ, replay memory $\mathcal{R}_i$, batch size $\mathcal{B}$
for episode $= 1$ to $\mathcal{A}$ **do**
  Reset the current satellite hidden state to the initial value
  Collect current satellite related state s_i^n and extract hidden state h_i'
  for $t = 1$ to T **do**
    Collect packet related state $s_{i,t}^p$ and current state $s_{i,t} = \{s_i^n, s_{i,t}^p\}$
    Determine the action $a_{i,t}$ according to the ϵ-greedy policy
    Send the packet k based on $a_{i,t}$
    Calculate the reward $r_{i,t}$ and $H_k = H_k - 1$
    Observe next state $s_{i,t+1}$
    Store transition $(s_{i,t}, a_{i,t}, r_{i,t}, s_{i,t+1})$ in $\mathcal{R}_i$
    if t mod $\mathcal{C} = 0$ **then**
      Update satellite related state s_i^n and extract hidden state h_i'
      Sample a random batch $\mathcal{B}$ transitions $(s_{i,t}, a_{i,t}, r_{i,t}, s_{i,t+1})$
      Calculate the loss based on (13)
      Update parameter θ_i based on (14)
    end if
  end for
  Reset $Q_i{}' = Q_i$
end for

where $RSHP(v_i, a_{i,t}, v_d)$ is defined in Section III-A, $\tau_{i,k,t}^q$ is queuing delay of packet k on v_i, $\tau_{i,j,t}^\eta$ is propagation delay between v_i and v_j, H_k is the TTL of packet k, ω_1, ω_2 and ω_3 are the weight factors, Ψ is a positive revenue when packet k reaches the destination successfully, and Φ is a negative penalty when packet k is discarded because the TTL decreases to zero or the sending queue is full. The closer the next hop is to the destination and the smaller the delay required to reach the next hop, the higher the reward.

C. The Training of GraphPR

As shown in Fig. 1, GraphPR has a GAT-enhanced Deep Q Network (DQN) architecture composed of a GAT and an FCNN. GAT encodes the satellite related states s_i^n and outputs a hidden state h_i', which is then concatenated with the packet related states s_i^p and fed into the FCNN. We train the GAT and FCNN in an end-to-end paradigm similar to [8]. Each agent i owns one policy network $Q_i(s_i, a_i; \theta_i)$ with parameter θ_i and one target network $Q_i'(s_i, a_i; \theta_i')$ with parameter θ_i'. The policy network of agent i consists of a GAT $\mathbb{G}_i(s_i^n; \theta_i^g)$ with parameter θ_i^g and a FCNN $\mathcal{F}_i(h_i'||s_i^p, a_i; \theta_i^f)$ with parameter θ_i^f, then the policy network $Q_i(s_i, a_i; \theta_i)$ can be also expressed as $\mathcal{F}_i(\mathbb{G}_i(s_i^n; \theta_i^g)||s_i^p, a_i; \theta_i^f)$, where $\theta_i = \{\theta_i^g, \theta_i^f\}$.

The detailed training procedure is shown in Algorithm 1. At the beginning of each episode, agent i firstly initializes the hidden state, then collects satellite related state s_i^n by observing the local states and sharing hidden states with neighbor satellites, and inputs it into GAT to extract the current hidden state h_i'. At the beginning of decision time slot t, packet k arrives at satellite i, and the agent collects the packet related state $s_{i,t}^p$ and combines it with s_i^n to form the current state $s_{i,t}$. Then the agent selects an action based on ϵ-greedy policy from the action space A according to the current state $s_{i,t}$.

$$a_{i,t} = \begin{cases} \text{a random action}, & \text{probability} = \epsilon \\ \text{argmax}_a Q_i(s_{i,t}, a_{i,t}; \theta_i), & \text{otherwise} \end{cases} \tag{12}$$

Based on the selected action $a_{i,t}$, packet k will be forwarded to the corresponding next hop. Then, the TTL of packet k is reduced by one, and the agent i can derive the reward $r_{i,t}$ and observe the next state $s_{i,t+1}$ from the LEO satellite network environment. After that, the agent i will record the transition $(s_{i,t}, a_{i,t}, r_{i,t}, s_{i,t+1})$ in the replay memory $\mathcal{R}_i$. Every $\mathcal{C}$ time slot, the agent i updates the satellite related state s_i^n by observing the local states and sharing hidden states with neighbor satellites, and extracts the new current satellite hidden state, while a batch of transitions is randomly sampled from $\mathcal{R}_i$ for updating policy network parameter. We use the Smooth L1 loss function, which is more robust to outliers than the Mean Squared Error (MSE) loss function. It calculates the MSE when the absolute value of the difference between the target value $y_{i,t}$ and the predicted value $Q_i(s_{i,t}, a_{i,t}; \theta_i)$ is less than 1 and calculates the mean absolute error in other cases. The loss function $loss_{i,t}$ is:

$$loss_{i,t} = \begin{cases} 0.5(y_{i,t} - Q_i(s_{i,t}, a_{i,t}; \theta_i))^2, \\ \qquad \text{if } |y_{i,t} - Q_i(s_{i,t}, a_{i,t}; \theta_i)| < 1 \\ |y_{i,t} - Q_i(s_{i,t}, a_{i,t}; \theta_i)| - 0.5, \text{otherwise} \end{cases} \tag{13}$$

The target value $y_{i,t}$ can be defined as $y_{i,t} = r_{i,t} + \gamma \text{max}_{a_{i,t+1}} Q_i'(s_{i,t+1}, a_{i,t+1}; \theta_i')$ with γ is the discount factor. The policy network parameter θ_i can be updated as:

$$\theta_i \leftarrow \theta_i + \alpha \nabla_{\theta_i} loss_{i,t} \tag{14}$$

where α denotes the learning rate. At the end of each episode, the target network parameter θ_i' is updated by using the policy network parameter θ_i .

IV. Evaluation

A. Experiment Setup

We implement GraphPR using Python for the simulation experiments. Different inclined orbit constellations are employed to evaluate the performance gain of our proposed GraphPR. These constellations have the same orbital inclination (70°) and orbital altitude (570 km), referring to the data from the second orbital layer of Starlink [11]. We conducted a series of experiments to compare the effects of different network loads, different numbers of satellites, and different satellite speeds on the performance of GraphPR, where the satellite speed is changed by setting different orbital altitudes. Each experiment will simulate the satellite constellation running for 200 minutes, and the experimental result will be recorded every minute and the final result is the average of these records.

The other experimental parameter settings are as follows: the carrier frequency f is 23.28 GHz, the channel bandwidth B is 25 MHz, the boltzmann constant k_B is 1.38×10^{-23} J/K, the effective isotropic radiated power $P_t G_t$ is 41.5 dBW, the quality factor G_r/T is 8.8 dB/K, the packet size ϑ is 1500 Bytes, the satellite related state update period is 1 min, the survival time of a routing entry is 300 ms, the maximum TTL $H_{\max}$ is 30 hops, the weight factors λ_1 and λ_2 are 0.55 and 0.45, respectively, the weight factors ω_1, ω_2 and ω_2 are 0.6, 0.55, and 0.45, respectively, the maximum queue length $q_{\max}$ is 640, the positive revenue Ψ is 1, the negative penalty Φ is -1, the initial exploration parameter ϵ is 0.90, the replay memory capacity $\mathcal{R}$ is 30000, the batch size $\mathcal{B}$ is 2048, the learning rate α is 0.001, the discount factor γ is 0.90, and the hidden layer size F_C is 256.

Authorized licensed use limited to: Yonsei Univ. Downloaded on April 10,2026 at 02:15:54 UTC from IEEE Xplore. Restrictions apply.

B. Baseline Algorithms

We compare GraphPR with the following baseline algorithms in terms of packet loss rate, average delivery time, throughput, and average queuing length.

- DT-TTAR [1]: This algorithm models the dynamic network as a series of topology snapshots and collects global link costs for each snapshot to calculate the path centrally by using the Dijkstra algorithm.
- FDR-MARL [7]: This algorithm is a fully-distributed routing algorithm based on MADRL, and uses spatial position to calculate residual propagation delay to guide the routing selection.
- DQN-IR [8]: The algorithm is a decentralized routing algorithm based on DQN, which can select the link adaptively according to the information including spatial position and queuing delay of the surrounding satellites.

C. Results and Analysis

Complexity Analysis: The network architecture of GraphPR consists of a single-layer GAT and an FCNN consisting of 4 fully connected layers. Assuming that the input graph $\mathcal{G}_g = (\mathcal{V}_g, \mathcal{E}_g)$ of GAT is a subgraph composed of the current satellite and its one-hop neighbor satellites, and the input feature size and output feature size of GAT are both F_S, the time complexity of the GAT can be expressed as $O(|\mathcal{V}_g|F_S^2 + |\mathcal{E}_g|F_S)$ according to [9]. Assuming that the dimension of the packet related state is F_P, and the number of neurons in the four fully connected layers of FCNN is F_C, F_C, F_C, F_A, the time complexity of the FCNN can be expressed as $O((F_S + F_P)F_C + 2F_C^2 + F_C F_A)$. Therefore, the overall time complexity of GraphPR is the sum of the time complexity of GAT and FCNN. The DT-TTAR runs the Dijkstra algorithm on the global topology $\mathcal{G} = (\mathcal{V}, \mathcal{E})$. Assuming the Dijkstra algorithm uses a priority queue with time complexity of $O((|\mathcal{V}| + |\mathcal{E}|)log|\mathcal{V}|)$, the time complexity of DT-TTAR is $O(|\mathcal{V}|(|\mathcal{V}| + |\mathcal{E}|)log|\mathcal{V}|)$. We can see that the time complexity of GraphPR depends on the feature size and network architecture, while that of DT-TTAR is determined by the number of satellites. Therefore, when the satellite scale becomes larger, the time complexity of GraphPR would be lower than that of DT-TTAR. According to the statistics of the experiment results, when the number of satellites is 45, 70, and 120, the inference time of GraphPR is 4.50 ms, 4.60 ms, and 4.86 ms, respectively, while the inference time of DQN-IR is 1.00 ms, 1.01 ms, and 1.13 ms, respectively, and the inference time of FDR-MARL is 0.93. ms, 1.01 ms, and 1.01 ms, respectively, and the inference time of DT-TTAR is 12.08 ms, 13.08 ms, and 76.71 ms, respectively. The inference time of GraphPR is reduced by at least 62.75% compared to DT-TTAR with the increase in the number of satellites.

Communication Overhead Analysis: According to the experimental results, when the number of satellites is 45, 70, and 120, the communication overhead of GraphPR is 9.60 Kbps, 16.73 Kbps, and 33.79 Kbps, respectively, while the communication overhead of DQN-IR is 0.77 Kbps, 1.19 Kbps, and 2.05 Kbps, respectively, and the communication overhead of FDR-MARL is 0.19 Kbps, 0.30 Kbps, and 0.51 Kbps, respectively, and the communication overhead of DT-TTAR is 24.07 Kbps, 69.70 Kbps, and 261.54 Kbps, respectively. It can be found that the communication overhead of GraphPR is slightly higher than that of FDR-MARL and DQN-IR. This is because GraphPR needs to regularly share the hidden state with the neighbor satellites, but FDR-MARL and DQN-IR only collect very simple information from the neighbor satellites. DT-TTAR has the highest communication overhead because its central satellite needs to collect the information of all ISLs for each routing decision.

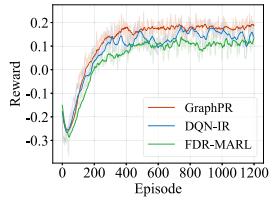

Fig. 2. Convergence analysis.

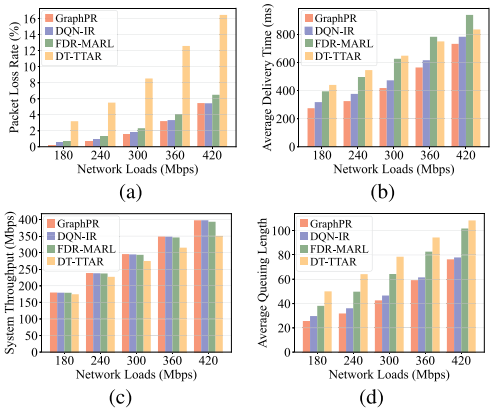

Fig. 3. Performance comparison with different network loads. (a) Packet loss rate. (b) Average delivery time. (c) System throughput. (c) Average queuing length.

Convergence Analysis: Fig. 2 shows the convergence results of FDR-MARL, DQN-IR, and GraphPR. All algorithms gradually become converged after 400 episodes. It can be found that GraphPR can achieve a better average reward than DQN-IR and FDR-MARL. The reason is that GraphPR can better perceive multi-hop information by sharing the GAT hidden states between satellites, which can avoid falling into local optimal solutions to some extent.

Performance Comparison: Fig. 3(a)–(d) illustrate the experimental results with different network loads. The number of satellites is fixed to 70. We can see that GraphPR can achieve 3.90%–91.54%, 6.47%–40.46%, 0.13%–13.19%, and 2.00%–50.4% performance improvement compared to baseline algorithms in terms of packet loss rate, average delivery delay, system throughput, and average queuing length. Only when the load is 420 Mbps, the packet loss rate and throughput of GraphPR are slightly worse than those of DQN-IR. This is because GraphPR tends to choose the path with a shorter distance and less load, and perceives multi-hop information through an information sharing mechanism and GAT representation learning to make better decisions. In addition, the packet loss rate of DT-TTAR increases rapidly as the network load increases, because it is difficult for DT-TTAR to obtain the link congestion information and link cost in time under high network load.

In Table I, the experimental results with different numbers of satellites are illustrated. The scenarios "num-45", "num-70", and "num-120" indicate that the number of satellites is 45, 70, and 120, respectively. The average communication capacity of a constellation will become larger when the number of satellites increases. Therefore, for the convenience of comparison, we set the network load as 120 Mbps, 240 Mbps, and 600 Mbps for the constellation with 45, 70, and 120 satellites, respectively. It can be observed that when the number of satellites is 45, the packet loss rate and throughput of GraphPR are slightly worse than the FDR-MARL. In other cases, GraphPR can achieve 26.47%–94.07%,

Authorized licensed use limited to: Yonsei Univ. Downloaded on April 10,2026 at 02:15:54 UTC from IEEE Xplore. Restrictions apply.

TABLE I
PERFORMANCE COMPARISON WITH DIFFERENT NUMBERS OF SATELLITES AND ORBITAL ALTITUDES

Metrics	Scenarios	GraphPR	DQN-IR	FDR-MARL	DT-TTAR
Packet Loss Rate (%)	num-45	0.61	0.98	0.31	2.33
	num-70	0.75	1.02	1.30	5.51
	num-120	0.89	1.73	5.38	15.00
	alt-570	0.58	1.47	1.11	4.23
	alt-970	1.84	1.75	1.79	6.58
	alt-1370	2.10	3.05	3.18	9.18
Average Delivery Time (ms)	num-45	377.51	448.12	517.40	539.77
	num-70	324.51	375.82	496.01	545.02
	num-120	322.29	347.00	461.86	512.91
	alt-570	336.58	392.71	510.72	556.52
	alt-970	419.20	499.65	637.39	675.52
	alt-1370	606.52	713.06	883.14	820.19
System Throughput (Mbps)	num-45	119.41	118.97	119.77	117.35
	num-70	238.49	237.83	237.18	227.05
	num-120	594.72	589.70	567.78	510.06
	alt-570	248.58	246.35	247.24	239.43
	alt-970	245.42	245.65	245.54	233.57
	alt-1370	244.76	242.40	242.06	227.06
Average Queuing Length	num-45	28.68	32.16	38.02	45.02
	num-70	31.84	36.12	49.77	64.19
	num-120	42.12	46.08	74.58	94.53
	alt-570	31.25	37.85	51.15	62.47
	alt-970	37.39	42.60	58.09	70.95
	alt-1370	49.45	55.84	72.70	80.33

7.12%–40.46%, 0.28%–16.6% and 8.59%–55.44% performance improvement compared with baseline algorithms in terms of packet loss rate, average delivery delay, system throughput and average queuing length. This is because the multi-hop information (indirectly) obtained by GAT can mitigate the influence of local observation, and improve the generalization ability of GraphPR to achieve better performance as the scale of the satellite network grows.

In Table I, the experimental results with different orbital altitudes are also presented. The scenarios "alt-570", "alt-970", and "alt-1370" mean that the orbital altitude is 570 km, 970 km, and 1370 km, respectively, and the corresponding satellite speed is 7.58 km/s, 7.37 km/s, and 7.17 km/s, respectively. The network load is fixed to 250 Mbps. It can be observed that except when the orbital altitude is 570 km, where the packet loss rate and throughput of GraphPR are slightly worse than the optimal values, GraphPR can achieve 31.15%–86.29%, 14.29%–39.52%, 0.54%–7.8%, and 11.44%–49.98% performance improvement compared with baseline algorithms in terms of packet loss rate, average delivery delay, system throughput, and average queuing length, this is because that GraphPR can better adapt to the dynamic changes of the satellite network through the distributed architecture and the multi-hop information extracted by GAT.

Comparison of Different GNN Models with Attention Mechanism: We replaced GAT in GraphPR with AGNN [12] and DotGAT [13] that apply attention mechanism. The network load is set to 300 Mbps and the number of satellites is 70. AGNN is an attention-based GNN, which calculates attention coefficients based on the similarity between node features. DotGAT applies a dot-product version of self-attention. The experimental results show that the packet loss rates for GAT, AGNN, and DotGAT are 1.58%, 1.71%, and 2.38%, respectively, the corresponding transmission times are 417.43 ms, 477.36 ms, and 477.00 ms, respectively, the corresponding throughputs are 295.62 Mbps, 295.23 Mbps, and 293.21 Mbps, respectively, and the corresponding average queue lengths are 42.62, 48.52, and 50.71, respectively. GraphPR with GAT can achieve a lower packet loss rate, smaller average delivery delay, shorter queuing length, and the highest system throughput, because GAT uses a self-attention mechanism to capture more complex node relationships and feature patterns.

V. CONCLUSION

In this paper, we study the dynamic packet routing problem in the LEO satellite networks, and propose the GraphPR algorithm, which constructs a fully distributed routing optimization framework using MADRL and utilizes GAT in MADRL to encode and share the perceived one-hop information. In addition, a special mechanism called RSPH is designed to guide the routing selection and avoid routing loops. The experimental results demonstrate that GraphPR can achieve lower average delivery time and packet loss rate, while keeping a higher throughput compared to the baseline algorithms.

REFERENCES

[1] J. Wenjuan and Z. Peng, "A discrete-time traffic and topology adaptive routing algorithm for leo satellite networks," *Int. J. Commun., Netw. System Sci.*, vol. 4, no. 1, pp. 42–52, 2011.
[2] Y.-H. Hsu, J.-I. Lee, and F.-M. Xu, "A deep reinforcement learning based routing scheme for leo satellite networks in 6G," in *Proc. 2023 IEEE Wireless Commun. Netw. Conf.*, 2023, pp. 1–6.
[3] P. Kumar, S. Bhushan, D. Halder, and A. M. Baswade, "fybrrlink: Efficient QoS-aware routing in SDN enabled future satellite networks," *IEEE Trans. Netw. Service Manag.*, vol. 19, no. 3, pp. 2107–2118, Sep. 2022.
[4] C. Li, W. He, H. Yao, T. Mai, J. Wang, and S. Guo, "Knowledge graph aided network representation and routing algorithm for LEO satellite networks," *IEEE Trans. Veh. Technol.*, vol. 72, no. 4, pp. 5195–5207, Apr. 2023.
[5] T. Pan, T. Huang, X. Li, Y. Chen, W. Xue, and Y. Liu, "OPSPF: Orbit prediction shortest path first routing for resilient LEO satellite networks," in *Proc. 2019 IEEE Int. Conf. Commun.*, 2019, pp. 1–6.
[6] X. Zhang, Y. Yang, M. Xu, and J. Luo, "Aser: Scalable distributed routing protocol for LEO satellite networks," in *Proc. IEEE 46th Conf. Local Comput. Netw.*, 2021, pp. 65–72.
[7] G. Xu, Y. Zhao, Y. Ran, R. Zhao, and J. Luo, "Spatial location aided fully-distributed dynamic routing for large-scale LEO satellite networks," *IEEE Commun. Lett.*, vol. 26, no. 12, pp. 3034–3038, Dec. 2022.
[8] P. Zuo, C. Wang, Z. Yao, S. Hou, and H. Jiang, "An intelligent routing algorithm for LEO satellites based on deep reinforcement learning," in *Proc. IEEE 94th Veh. Technol. Conf. (VTC2021-Fall)*, 2021, pp. 1–5.
[9] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Proc. Int. Conf. Learn. Representations*, 2018. [Online]. Available: https://openreview.net/forum?id=rJXMpikCZ
[10] F. Huang et al., "Design and implementation of the inter-satellite link budget software," *J. Softw. Eng.*, vol. 9, pp. 230–241, 2015.
[11] S. Ren, X. Yang, R. Wang, S. Liu, and X. Sun, "The interaction between the LEO satellite constellation and the space debris environment," *Appl. Sci.*, vol. 11, no. 20, 2021, Art. no. 9490.
[12] K. K. Thekumparampil, C. Wang, S. Oh, and L.-J. Li, "Attention-based graph neural network for semi-supervised learning," 2018, *arXiv:1803.03735*.
[13] Y.-C. Chung et al., "A domain adaptation approach for resume classification using graph attention networks and natural language processing," *Knowl.-Based Syst.*, vol. 266, 2023, Art. no. 110364.

Authorized licensed use limited to: Yonsei Univ. Downloaded on April 10,2026 at 02:15:54 UTC from IEEE Xplore. Restrictions apply.